

Report di Analisi Statica del Malware

notepad-classico.exe

Corso di Cybersecurity – Epicode

August 4, 2026

1 Introduzione

Il presente report descrive i risultati dell'analisi statica condotta sul file eseguibile **notepad-classico.exe**, individuato all'interno della cartella **Malware** della macchina virtuale **FlareVM**. L'analisi è stata effettuata tramite lo strumento **PEStudio**, che consente di ispezionare la struttura del formato Portable Executable (PE) senza eseguire il file.

L'obiettivo dell'analisi è duplice:

- individuare e descrivere le **librerie importate** dal file;
- individuare e descrivere le **sezioni** di cui si compone il PE.

Informazioni generali sul file

Proprietà	Valore
Nome file	notepad-classico.exe
Percorso	c:\users\flarevm\desktop\malware\
Tipo	Eseguibile, 32-bit, GUI
Dimensione	289280 byte
Entropia file	6.273
SHA-256	D2E6C9F9273663F3218BCD7CBFB3B6F599FBCE7A4BA986F9BBFF77E3603988F
Nome file originale (version info)	NOTEPAD.EXE
Descrizione (version info)	Blocco note
Compilatore	Microsoft Linker 1.0

2 Librerie Importate

Il file importa un totale di **9 librerie** (DLL), per un totale di **201 funzioni** importate, di cui **18** vengono segnalate da PEStudio come potenzialmente degne di attenzione (flag). Di seguito viene fornita, per ciascuna libreria, una descrizione della sua funzione tipica all'interno del sistema operativo Windows, inoltre viene riportato come confronto lo screenshot delle librerie trovate.

Libreria	N. im-port	Descrizione
COMDLG32.dll	9	Libreria delle finestre di dialogo comuni di Windows (Common Dialogs Library). Fornisce le funzionalità standard per l'apertura/salvataggio dei file, la stampa, la scelta dei font e la ricerca/sostituzione del testo – funzionalità tipiche di un editor di testo.
SHELL32.dll	4	Windows Shell Library. Fornisce funzioni di interazione con la shell del sistema operativo, tra cui la gestione delle icone e il supporto al drag-and-drop dei file da Esplora Risorse verso l'applicazione.
WINSPOOL.DRV	3	Windows Spooler Driver. Fornisce l'interfaccia con lo spooler di stampa di Windows, necessaria per l'invio dei documenti in stampa.
COMCTL32.dll	1	Common Controls Library. Contiene i controlli comuni dell'interfaccia utente di Windows (barre di stato, toolbar, ecc.); in questo caso utilizzata per la barra di stato dell'applicazione.
msvcrt.dll	22	Microsoft C Runtime Library. Fornisce le funzioni standard del linguaggio C (gestione della memoria, delle stringhe, del tempo, terminazione del processo). È presente praticamente in ogni eseguibile compilato in C/C++.
ADVAPI32.dll	7	Advanced Windows 32 Base API. Espone funzionalità avanzate del sistema, in particolare l'accesso al registro di sistema (apertura, lettura, creazione e scrittura di chiavi) e la gestione dei privilegi. È una libreria da monitorare con attenzione in ambito malware, poiché la manipolazione del registro è una tecnica comune per ottenere persistenza.
KERNEL32.dll	57	Windows NT BASE API Client. Libreria fondamentale del sistema operativo: espone le funzioni core per la gestione di file, memoria, processi e moduli (caricamento dinamico di librerie tramite LoadLibrary , risoluzione di funzioni tramite GetProcAddress , ecc.). Presente in qualsiasi eseguibile Windows.
GDI32.dll	24	GDI Client Library (Graphics Device Interface). Fornisce le funzioni per il disegno a video di testo, font e oggetti grafici, utilizzata per il rendering dell'interfaccia grafica dell'applicazione.
USER32.dll	74	Multi-User Windows USER API Client Library. Gestisce la creazione e la gestione delle finestre, dei menu, dei messaggi di sistema e degli eventi da tastiera/mouse; è alla base di qualsiasi applicazione dotata di interfaccia grafica (GUI).

library (9)	flag (0)	type	imports (201)	description
comdlg32.dll	-	Implicit	<u>9</u>	Common Dialogs Library
SHELL32.dll	-	Implicit	<u>4</u>	Windows Shell Library
WINSPool.DRV	-	Implicit	<u>3</u>	Windows Spooler Driver
COMCTL32.dll	-	Implicit	<u>1</u>	Common Controls Library
msvcrt.dll	-	Implicit	<u>22</u>	Microsoft C Runtime Library
ADVAPI32.dll	-	Implicit	<u>7</u>	Advanced Windows 32 Base API
KERNEL32.dll	-	Implicit	<u>57</u>	Windows NT BASE API Client
GDI32.dll	-	Implicit	<u>24</u>	GDI Client Library
USER32.dll	-	Implicit	<u>74</u>	Multi-User Windows USER API Client Library

Figure 1: Confronto tra le librerie estratte dal ia e lo screenshot delle librerie

Funzioni segnalate (flag = 18)

PEStudio segnala automaticamente 18 funzioni importate come potenzialmente sospette, in quanto storicamente associate a comportamenti malevoli o comunque degne di approfondimento. Tra queste si segnalano in particolare:

- **LoadLibraryA / GetProcAddress** – caricamento dinamico di librerie e risoluzione di funzioni a runtime; tecnica spesso impiegata per offuscare le reali API utilizzate dal programma, evitando che compaiano staticamente nella import table.
- **RegCreateKeyW / RegSetValueExW** – creazione e scrittura di chiavi nel registro di Windows; comunemente associate a meccanismi di persistenza.
- **CreateFileMappingW / MapViewOfFile / UnmapViewOfFile** – mappatura di file/memoria; tecnica utilizzata anche per l'iniezione di codice in altri processi.
- **FindFirstFileW / DeleteFileW / WriteFile** – enumerazione, cancellazione e scrittura di file sul filesystem.
- **TerminateProcess / GetCurrentProcess** – terminazione di processi.

3 Sezioni del PE

Il file è composto da **5 sezioni**. Nella tabella seguente vengono riportate le caratteristiche principali di ciascuna, seguite da una descrizione del loro ruolo tipico e, dove rilevante, delle anomalie riscontrate.

Sezione	Entropia	Descrizione
.text (section[0])	6.214	Sezione standard contenente il codice eseguibile (istruzioni macchina) del programma. Permessi execute . Contiene la import directory e il base-of-code . Entropia coerente con codice compilato normale. Nessuna anomalia rilevata.
.data	1.149	Contiene i dati inizializzati del programma (variabili globali/statiche con valore iniziale). Permessi read+write . Entropia molto bassa, coerente con dati strutturati non compressi/cifrati. Nessuna anomalia rilevata.

Sezione	Entropia	Descrizione
.rsrc	5.421	Sezione delle risorse dell'applicazione: icone, stringhe, finestre di dialogo, informazioni di versione e manifest. Permessi di sola lettura. Dimensione compatibile con le 23 risorse rilevate da PESTudio. Nessuna anomalia rilevata.
.text (section[3])	6.428	Sezione anomala. Si tratta di una <i>seconda</i> sezione denominata <code>.text</code> , condizione non tipica di un eseguibile legittimo, nel quale il codice risiede normalmente in un'unica sezione <code>.text</code> . Presenta permessi <code>read+write+execute</code> (RWX) ed è marcata come self-modifying . Inoltre, l' entry point del programma è collocato proprio all'interno di questa sezione, anziché nella prima <code>.text</code> .
.idata	5.439	Contiene la Import Address Table (IAT), ovvero i riferimenti alle funzioni importate dalle librerie esterne descritte nella Sezione 1. Permessi <code>read+write</code> .

Osservazioni sulla sezione anomala

La presenza di una seconda sezione `.text` con permessi RWX, flag *self-modifying* attivo ed entry point del programma spostato al suo interno costituisce un forte indicatore che il file sia stato **impacchettato (packed)** o comunque contenga un componente (stub/loader) in grado di decifrare o generare codice a runtime. Questo pattern è comunemente utilizzato per ostacolare l'analisi statica e l'individuazione da parte degli strumenti antivirus, poiché il payload reale non è direttamente visibile nel codice presente su disco, ma viene ricostruito solo durante l'esecuzione.

4 Considerazione Finale

L'analisi statica condotta su `notepad-classico.exe` porta a formulare alcune considerazioni sulla natura del file, pur trattandosi di un'ipotesi che andrebbe confermata da un'analisi dinamica (esecuzione controllata in ambiente sandbox).

A un primo sguardo il file si presenta come un normale eseguibile Windows: le informazioni di versione lo identificano come `NOTEPAD.EXE` (*Blocco note*), il manifest fa riferimento a `Microsoft.Windows.Shell.notepad`, e la maggior parte delle librerie importate (`USER32.dll`, `GDI32.dll`, `COMDLG32.dll`, `COMCTL32.dll`, `SHELL32.dll`, `WINSPOOL.DRV`) è del tutto coerente con un semplice editor di testo dotato di interfaccia grafica. Questo suggerisce un tentativo di **mascheramento (masquerading)**: il file cerca di apparire, sia nei metadati sia nel comportamento superficiale, come l'applicazione legittima di Windows.

Tuttavia, l'analisi della struttura del PE rivela elementi che si discostano significativamente da questo profilo:

- **Import sospetti.** Tra le funzioni importate figurano `LoadLibraryA` e `GetProcAddress`, tipicamente usate per risolvere API a runtime ed evitare che compaiano in chiaro nella import table; funzioni di accesso al registro (`RegCreateKeyW`, `RegSetValueExW`), spesso associate a meccanismi di persistenza; e funzioni di mappatura della memoria (`CreateFileMappingW`, `MapViewOfFile`, `UnmapViewOfFile`), che possono essere impiegate per tecniche di iniezione di codice.

- **Sezione .text duplicata e anomala.** La presenza di una seconda sezione denominata `.text`, con permessi `read+write+execute` e flag *self-modifying* attivo, unita al fatto che l'**entry point** del programma risieda proprio in questa sezione anziché nella prima, è un indicatore tipico di **packing** o di codice che si decifra/genera a runtime.

La combinazione di questi elementi – un contenitore che si presenta come applicazione legittima e nota, ma il cui reale punto di ingresso si trova in una sezione scrivibile ed eseguibile con caratteristiche di auto-modifica – è coerente con lo scenario di un **eseguibile impacchettato o infetto**, in cui il payload malevolo reale è celato e viene decompresso/decifrato solo al momento dell'esecuzione, così da eludere l'analisi statica e le firme antivirus basate sul contenuto del file su disco.

In conclusione, sulla base della sola analisi statica, `notepad-classico.exe` presenta caratteristiche compatibili con un **trojanizzazione** di un'applicazione legittima (Blocco note) oppure con l'impiego di un **packer/protector** per nascondere un payload malevolo. Per confermare tale ipotesi e determinare il comportamento effettivo del file (es. comunicazioni di rete, scrittura di file aggiuntivi, modifiche al registro, creazione di processi figli) sarebbe necessaria un'**analisi dinamica** in ambiente isolato e monitorato.